StudyAI

An AI-Powered Study and Examination Preparation System

A web application that converts a student's own study material into summaries, key points, mind maps, flashcards, practice questions, timed examinations and an oral viva — grounded entirely in the uploaded documents and built on free services.

PROJECT SYNOPSIS

Project title	StudyAI — AI-Powered Study and Examination Preparation System
Domain	Artificial Intelligence, Natural Language Processing, Web Development
Application type	Client–server web application
Front end	HTML5, CSS3, vanilla JavaScript (no framework, no build step)
Back end	Python 3.10+, Flask, SQLite
AI approach	Retrieval-Augmented Generation over free LLM providers

1. Introduction

Students preparing for examinations spend a large share of their time on mechanical work rather than on learning: condensing chapters into notes, writing flashcards, hunting for practice questions and building revision timetables. General-purpose AI chatbots can perform some of these tasks, but they answer from their own training data. For a student this is a serious limitation — the answer may not match the prescribed syllabus, may contradict the textbook, and cannot be verified against the material the student will actually be examined on.

StudyAI addresses this by inverting the relationship. The student uploads their own material — chapter PDFs, typed notes, a CSV of data, or even a recorded lecture — and every feature of the system is generated strictly from that material. Each answer carries a citation back to the document, heading and page it came from, so a claim can always be checked against the source. Where the material does not contain an answer, the system says so explicitly instead of inventing one.

The entire system runs on free services. It requires no paid API subscription, no vector database and no cloud account, which makes it deployable by an individual student on a personal computer.

2. Problem Statement

Existing approaches to AI-assisted study fail students in four specific ways:

- **Ungrounded answers.** A general chatbot answers from training data, not from the student's syllabus, and offers no way to verify a claim.
- **Hallucination.** When a model does not know something it commonly invents a plausible answer, which is worse than no answer when a student is revising for an examination.
- **Context-free output.** Automatically generated notes frequently produce fragments such as "This is the core technology", where the subject of the sentence is stranded in a line the reader cannot see. Such a point is useless on a flashcard or in a revision list.
- **Cost and reliability.** Reliable models are paid, while free tiers impose strict daily limits, so a system built on a single provider stops working without warning.

The objective of this project is a study system that is grounded, verifiable, resistant to hallucination, readable without surrounding context, and free to operate.

3. Objectives

- To accept study material in multiple formats (PDF, TXT, CSV, audio lecture, or a web link) and prepare it automatically for study.
- To generate a chapter summary, key points, definitions, a visual mind map, flashcards, practice questions and a full timed examination paper from that material.
- To answer a student's questions using only the uploaded documents, citing the exact source of every claim.
- To conduct an oral viva in which the system asks the questions, marks the student's typed answers out of ten and gives written feedback.
- To guarantee that every generated sentence is self-contained, factually traceable to the material, and free of unresolved pronouns.
- To remain operational on free services by failing over automatically between multiple AI providers, and to degrade gracefully to a local extractive mode when none is reachable.

- To track a student's performance over time and produce an automatic progress report identifying weak topics.

4. Proposed System

StudyAI is organised as a three-stage pipeline. Material is first **ingested** and indexed, then **retrieved** selectively when a generation is requested, and finally **verified** before any output reaches the student.

4.1 Document ingestion

Uploaded files are converted to text and cleaned. PDF extraction is a particular problem because a line break inside a sentence is an artefact of page layout, not a paragraph boundary; left uncorrected it fragments every sentence in the document. The extractor rejoins mid-sentence line breaks, repairs hyphenated words split across lines, and removes running headers and footers that repeat across pages.

The cleaned text is divided into retrieval units by a heading-aware semantic chunker. A chunk never crosses a heading and never splits a sentence; it records the heading and page number it came from, and repeats the closing sentences of the preceding chunk so that a fact spanning a boundary remains retrievable from either side.

4.2 Retrieval-Augmented Generation

Rather than sending an entire document to the model, the system retrieves only the passages relevant to the request. Candidate chunks are ranked by BM25, which weights rare terms and normalises for length, and the shortlist is then re-scored on how much of the question it actually covers, how close the matched terms appear, and whether the section heading matches. The neighbouring chunks of each winner are included, because a definition and its explanation often sit either side of a boundary. Finally each passage is compressed to the sentences that carry question terms, so that several distinct sources fit within the prompt instead of one long extract.

Every passage is labelled with its document, heading and page. These labels are given to the model, which cites them inline, and are shown to the student beneath the answer.

4.3 Output verification

Prompt instructions alone do not guarantee quality from free models, so every generated output passes through a verification stage before display. This stage detects a sentence that opens with an unresolved pronoun and repairs it by substituting the grammatical subject of the preceding sentence; where no subject can be recovered the line is discarded. It also removes fragments, near-duplicate points, and statements that are grammatical but carry no information.

5. Illustration of Output Verification

The following example shows the verification stage operating on raw model output. Two lines are repaired, two are discarded, and one is retained unchanged.

Raw generated line	Action	Final output
This – the core technology.	Pronoun resolved from previous sentence	Retrieval-Augmented Generation – the core technology.
It is used for retrieval.	Pronoun resolved	Retrieval-Augmented Generation is used for retrieval.
These are important.	Discarded — no information content	—
They	Discarded — sentence fragment	—
RAG combines search with generation.	Retained unchanged	RAG combines search with generation.

Table 1 — Behaviour of the output verification stage.

6. System Modules

Module	Function
Document ingestion	Extracts text from PDF, TXT, CSV, audio and web links; cleans layout artefacts; performs heading-aware semantic chunking.
Retrieval engine	Hybrid BM25 and phrase-proximity ranking, reranking, neighbour expansion, context compression and source attribution.
Provider manager	Routes each request to the healthiest free AI provider; benches a failing provider with exponential backoff and restores it automatically on recovery.
Quality gate	Resolves pronouns, removes fragments, duplicates and content-free statements from every generated output.
Analysis generator	Produces the chapter summary, key points, definitions and mind map; results are cached so material is processed once.
Assessment generator	Produces flashcards, six types of practice question, and a sectioned examination paper with server-side grading.
Viva examiner	Asks spoken-style questions one at a time, marks each typed answer out of ten and returns written feedback and a model answer.
Progress reporting	Records attempts and per-question timing, detects weak topics and generates a revision plan.
Support tools	Sticky-note board, scientific calculator with its own expression parser, text-to-speech, and previous-year paper search.

Table 2 — Functional decomposition of the system.

7. Reliability of Free Services

Free AI tiers impose daily and per-minute request limits, and model identifiers are retired without notice. A system depending on one provider therefore fails unpredictably. StudyAI addresses this with a provider manager that maintains a health record for each configured service.

Requests are routed to the highest-priority healthy provider. A provider that returns a rate-limit response, times out, or errors is placed in a cooldown that doubles with each consecutive failure, from thirty seconds to a ceiling of fifteen minutes; the next provider serves the request transparently. A single successful response clears the record and returns the provider to normal priority, so recovery requires no intervention. When no provider is reachable the system falls back to a local extractive mode that continues to produce summaries, key points and cited answers directly from the documents.

Priority	Provider	Role
1	Google Gemini (free tier)	Primary — highest free request limits
2	OpenRouter (free models)	Secondary — multiple open models
3	Groq (free tier)	Tertiary — lowest latency
4	Hugging Face Inference	Final network fallback
5	Local extractive mode	Offline operation; requires no key

Table 3 — Provider failover chain.

8. Technology Stack

Layer	Technology	Justification
Front end	HTML5, CSS3, JavaScript (ES2020)	No framework or build step, so the application runs directly from source.
Back end	Python 3.10+, Flask	Lightweight routing suited to a service-oriented backend.
Database	SQLite (WAL mode)	Serverless, single file, sufficient for per-user document and progress data.
Retrieval	BM25 implemented in Python	Requires no embedding service or vector database, keeping the system free.
AI providers	Gemini, OpenRouter, Groq, Hugging Face	All offer free tiers; used interchangeably through one interface.
Document parsing	pypdf	Pure-Python PDF text extraction with no external binary dependency.
Speech to text	Whisper (local, optional)	Lecture transcription performed on the user's own machine.
Text to speech	Browser SpeechSynthesis API	Built into the browser; requires no service.
Security	PBKDF2-HMAC-SHA256, HttpOnly cookies	Passwords stored only as salted hashes; session tokens unreadable by scripts.

Table 4 — Technology selection and rationale.

9. System Requirements

9.1 Software

- Python 3.10 or later
- Flask, pypdf, requests, python-dotenv (installed via requirements.txt)
- A modern web browser (Chrome, Edge or Firefox)
- At least one free AI provider key; optional, as the system runs without one

9.2 Hardware

- Processor: dual-core 2.0 GHz or higher
- Memory: 4 GB RAM (8 GB recommended if local speech transcription is used)
- Storage: 500 MB, plus space for the SQLite database
- An internet connection for AI features; local mode operates offline

10. Implementation Outcomes

The following results were obtained from the working implementation.

Aspect	Result

Perceived responsiveness	All views are generated in parallel on upload rather than on first click. Upload returns in approximately 0.4 s; the five generated views complete in the background in about 27 s.
Repeat access	Generated results are stored, so after a restart each view loads from disk in 50–270 ms and consumes no further AI quota.
Grounding	Answers cite the document, heading and page used, for example "physics-ch3.pdf › Ohm's Law › page 1".
Output quality	Across all generated views, no bullet begins with an unresolved pronoun; verification is applied to AI and local output alike.
Fault tolerance	Verified by fault injection: a failing provider is benched, the next serves transparently, backoff doubles per failure, and one success restores the provider.
Continuity without AI	With every provider unavailable, the full feature set was exercised successfully in local extractive mode.

Table 5 — Measured outcomes of the implementation.

11. Future Scope

- Optical character recognition, to support scanned and photographed textbook pages which cannot presently be read.
- Semantic retrieval using sentence embeddings, to match questions phrased differently from the source text.
- Spaced-repetition scheduling of flashcards based on recorded performance.
- Support for diagrams and images extracted from the source material.
- A shared classroom mode allowing a teacher to distribute material and review aggregate performance.
- Native mobile applications for offline revision.

12. Conclusion

StudyAI demonstrates that a dependable AI study assistant can be built without paid infrastructure, provided the system is designed around the limitations of free services rather than in spite of them. Grounding every output in the student's own material, and citing its source, addresses the verification problem that makes general chatbots unsuitable for examination preparation. Verifying output after generation, rather than trusting the prompt, addresses the quality problem that affects smaller free models. Automatic failover across several providers, with a local extractive fallback, addresses the reliability problem inherent in free tiers. Together these decisions produce a system that a student can depend on.

13. References

- Lewis, P. et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* Advances in Neural Information Processing Systems 33.
- Robertson, S. and Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond.* Foundations and Trends in Information Retrieval, 3(4).
- Radford, A. et al. (2022). *Robust Speech Recognition via Large-Scale Weak Supervision.* (Whisper.)
- Flask documentation — https://flask.palletsprojects.com
- Google AI Studio documentation — https://ai.google.dev
- SQLite documentation — https://sqlite.org/docs.html